Neovim debugging — nvim-dap

C++ / C / Rust use **codelldb** · Python uses **debugpy** · same keys and UI for all · <leader> = Space

Start a session

1. Put the cursor on a line, **<leader>db** sets a breakpoint.
2. **<F5>** and pick a configuration. dap-ui opens by itself.
3. Step / inspect with the keys below. **<leader>dq** stops and closes the UI.
4. **<leader>dl** re-runs the last configuration without the picker.

Run & step

<F5>	Start (pick config) / continue to next breakpoint
<F10>	Step over
<F11>	Step into (Windows Terminal may grab F11 for fullscreen)
<F12>	Step out
<leader>dc	Run to cursor
<leader>dl	Run last configuration
<leader>dq	Terminate session, close dap-ui

Breakpoints & inspect

<leader>db	Toggle breakpoint on current line
<leader>dB	Conditional breakpoint (asks for expression)
<leader>de	Eval word under cursor (visual: the selection)
<leader>dr	Open REPL: evaluate expressions while stopped
<leader>du	Toggle dap-ui panels

Python only

<leader>dt	Debug the test method under the cursor (pytest)
<leader>dT	Debug the test class under the cursor
<leader>ds	Visual mode: debug the selected code

dap-ui layout

• **Left:** Scopes (variables) · Breakpoints · Stacks · Watches
• **Bottom:** REPL · Console (program output / `input()`)
• In a panel: **<CR>** expand / jump, **e** edit value, **d** delete (breakpoints, watches)

Built-in configurations

C / C++
• Launch: asks for the executable path. Build with `-g`.

Python — runs with `$VIRTUAL_ENV`, else `./.venv`
• Debug file: current file, like `python file.py`
• Debug file with args · step into libraries (justMyCode off)
• Debug module: `python -m pkg.mod` (pre-filled)
• Debug pytest (current file): `pytest file -x`
• Attach: start `python -m debugpy --listen 5678 --wait-for-client f.py`, then attach

Rust: cargo-built binary / tests / examples, manual path, attach.

Per-project: .vscode/launch.json

• Read from <cwd>`/.vscode/launch.json` on every **<F5>** — start nvim from the project root.
• VS Code format: comments, `${file}`, `${workspaceFolder}`, `${env:VAR}`.
• `"type"`: `codelldb` (C++/Rust), `debugpy` or `python`.

```
{ "configurations": [
  { "type": "codelldb", "request": "launch",
    "name": "my_app",
    "program": "build/my_app",
    "args": ["--flag"],
    "cwd": "${workspaceFolder}" },
  { "type": "debugpy", "request": "launch",
    "name": "script", "program": "${file}",
    "console": "integratedTerminal" }
] }
```

Setup checklist

• Debuggers via Mason: `codelldb`, `debugpy` (auto-install, or `:MasonInstall`). Needs VPN (Artifactory).
• C++: compile with `-g` (Debug / RelWithDebInfo), else no breakpoints.
• Python tests: `pytest` inside the venv (`uv add --dev pytest`) + treesitter parser (`:TSInstall python`).
• No attach-by-PID: `ptrace_scope=1`. Use launch or `--listen`.

Troubleshooting

• Breakpoint shown but never hit: binary not built with `-g`, or stale build.
• `:DapShowLog` for adapter errors · `:checkhealth dap`
• Panels stuck open: **<leader>du** toggles them.